Webprogramozás

Kvíz tananyaga

Eötvös Loránd Tudományegyetem

2026-2027/1



Projekt felépítése

- `index.html`, amely tartalmazza:

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <script src="script.js" defer></script>
</body>
```

- `style.css`
- `script.js`

Első program: Helló Világ!

```
console.log("Helló Világ!");
```

A weblap megnyitásakor a böngésző konzoljában megjelenik a szöveg.

- Chromium alapú böngészőkben: `Ctrl + Shift + J`
- Firefox: `Ctrl + Shift + K`

Változók

```
let x = 5;     // blokkszintű változó
const y = 10; // konstans (inicializálandó)

var z = 15;    // függvényszintű/globális
               // változó (ELAVULT!)
```

Típusok és literáljai

- **Primitív típusok:**
  - `Boolean`: `true`, `false`
  - `null`: érték nélküli változó
  - `undefined`: inicializálatlan változó
  - `Number`: `5`, `3.14`, `NaN`, `Infinity`
  - `BigInt`: `9007199254740991n`
  - `String`: `"Hello World!"`, `'Hello World!'`, `""`, `` `Hello ${name}!` `` (ahol `name` egy `String` kifejezés)
  - `Symbol`: egyedi azonosító
- **Összetett típusok:**
  - `object`: `{ key1: "value1", key2: "value2" }`
  - `array`: `[1, 2, 3, 4, 5]`, `[]`
  - `function`: `function myFunction() { return "Hello World!"; }`

Forrás: *MDN Web Docs*

Típusok és műveleteik

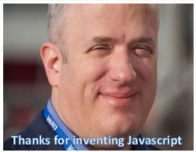
A típusokhoz tartozó műveletek nagyban hasonlítanak a már ismert C-szerű nyelvekhez:

- `Number` és `BigInt`: `+`, `-`, `*`, `/`, `%`, `**`, `++`, `--`
- `String`: `+` (összefűzés)
- `Boolean`: `&&`, `||`, `!`
- Összehasonlító operátorok: `==`, `!=`, `===`, `!==`, `<`, `>`, `<=`, `>=`, `instanceof`
- Hozzárendelő operátorok: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`

Forrás: *MDN Web Docs*

Típuskényszerítés

```
> typeof NaN
< "number"
> 9999999999999999
< 10000000000000000
> 0.5+0.1==0.6
< true
> 0.1+0.2==0.3
< false
> Math.max()
< -Infinity
> Math.min()
< Infinity
> []+[]
< ""
> []+{}
< "[object Object]"
> {}+[]
< 0
> true+true+true===3
< true
> true-true
< 0
> true==1
< true
> true===1
< false
> (!+[]+[]+![]).length
< 9
> 9+"1"
< "91"
> 91-"1"
< 90
> []==0
< true
```



Forrás: *Reddit*

- + operátor
  - `Number` + `Number` ⇒ összeadás
  - `Object` + _ ⇒ összeadás
  - ellenben ⇒ összefűzés, illetve operandusok szöveggé konvertálása (`toString()` fgv.)
- false értékek: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`
- `true` → `1`, `false` → `0`
- == : típuskonverziót végez,
  === : típusazonosságot is ellenőriz
- – operátor: mindig számként kezeli az operandusokat

Elágazások

- if - else if - else

```
if (x > 0) {
  console.log("x poz");
} else if (x < 0) {
  console.log("x neg");
} else {
  console.log("x == 0");
}
```

- Hármas operátor

```
let result = x % 2 === 0
  ? "páros"
  : "páratlan";
```

- switch - case - default

```
switch (x) {
  case 1:
    console.log("x == 1");
    break;
  case 2:
    console.log("x == 2");
    break;
  default:
    console.log("x egyik sem");
}
```

Ciklusok

- for ciklus

```
for (let i = 0; i < 10; i++) {
  console.log(i);
}
```

- while ciklus

```
while (i < 10) {
  ...
}
```

- do while ciklus

```
do {
  ...
} while (i < 10);
```

- for...in ciklus

```
const obj = { a: 1, b: 2, c: 3 };
for (const key in obj) {
  console.log(key, obj[key]);
}
```

- for...of ciklus

```
const arr = [1, 2, 3];
for (const value of arr) {
  console.log(value);
}
```

Függvények

- `function` kulcsszó

```
function add(a, b) {
  return a + b;
}
```

- lambda függvény (arrow function)

```
const add = (a, b) => a + b;
```

- többsoros lambda függvény

```
const add = function(a, b) {
  return a + b;
};
```

Mindegyik hívása azonos módon történik, pl.: `add(1, 2)`.

Tömbök

- Tetszőleges (dinamikus) mennyiségű és típusú elemet tartalmazhatnak:

```
let array = [false, 1, "ketto", [0, 1, 2]];
```

- 0-tól indexelt:

```
console.log(array[0]);     // false
console.log(array[3][0]); // 0
```

- Tárolják saját hosszuk: `array.length`
- Végére
  - beszúrni: `array.push(elem);`
  - eltávolítani: `array.pop();`
- Elejére
  - beszúrni: `array.unshift(elem);`
  - eltávolítani: `array.shift();`

Tömbfüggvények

Legyen `l` egy tetszőleges logikai kifejezés `array` tömb elemeire.
A tömbök osztálya (`Array`) számos egyéb, programozási tételekhez köthető metódust is implementál:

- `find()` - keresés

  ```
  array.find((elem) => l); // array egy eleme
  ```

- `filter()` - szűrés

  ```
  array.filter((elem) => l); // array elemeiből álló tömb
  ```

  eredménye egy újabb tömb ⇒ használható újabb `Array` metódus

- `every()` és `some()` - univerzális- és egzisztenciális kvantorok

  ```
  array.every((elem) => l); // igaz-e mindegyikre
  array.some((elem) => l);  // igaz-e bármelyikre
  ```

Tömbfüggvények

- `forEach()` - függvényszintű for-each ciklus

```
array.forEach((elem) => console.log(elem));
```

- `flat()` - többdimenziós tömb sorfolytonossá alakítása

```
[[1, 2], [3, 4, [5]]].flat(); // [1, 2, 3, 4, 5]
```

- `join()` - tömb elemeinek szöveggé fűzése

```
[1, 2, 3, 4].join(", "); // "1, 2, 3, 4"
```

- `map()` - tömb elemeinek transzformációja

```
[1, 2, 3, 4].map((elem) => elem * 2); // [2, 4, 6, 8]
[{a: 1, b: 2}, {a: 3, b: 4}].map((e) => e.a); // [1, 3]
```

- `reverse()` - tömb elemeinek megfordítása

```
[1, 2, 3, 4].reverse(); // [4, 3, 2, 1]
```

Tömbfüggvények

Reduktor függvény (`reduce()`) segítségével megadhatók további programozási tételek:

- összegzés

```
array.reduce((acc, curr) => acc + curr, 0);
```

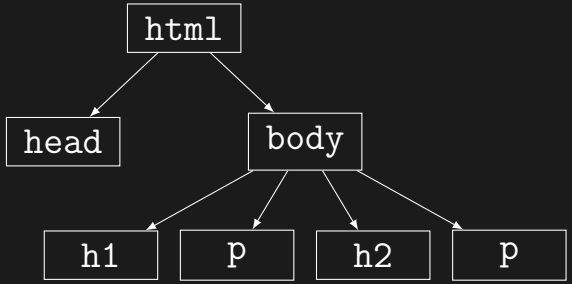
- maximumkiválasztás

```
array
  ? array.reduce((acc, curr) => Math.max(acc, curr))
  : null;
// vagy
Math.max(...array); // ... = spread operator
```

DOM - Document Object Model

Document **O**bject **M**odel az oldal HTML kódjának reprezentációja, melyben szerepel minden HTML elemnek megfelelő Javascript objektum.

```
<!DOCTYPE html>
<html>
  <head></head>
  <body>
    <h1>Cím</h1>
    <p>Szöveg</p>
    <h2>Alcím</h2>
    <p>Még több szöveg</p>
  </body>
</html>
```



DOM - Elemek elérése

Az elemek kiválasztásához CSS szelektorokat lehet használni.

Az első illeszkedő elemet a

```
document.querySelector('p');
```

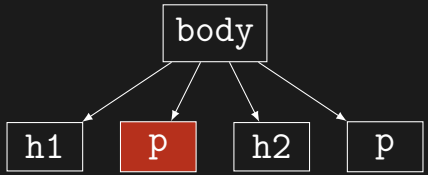
segítségével lehet kiválasztani.

Amennyiben nem illeszkedik egyetlen elem sem a szelektorra, akkor `null`-t ad vissza.

Egy elemnek egy adott CSS-szelektorra való illeszkedése megállapítható a

```
elem.matches('p'); // true or false
```

segítségével.

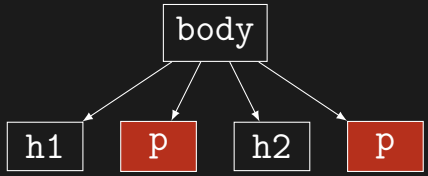


DOM - Elemek elérése

Az összes illeszkedő elemet a
`document.querySelectorAll('p');`
segítségével lehet kiválasztani.

Ez egy iterálható lista pl.
`for (let elem of allElems)`
segítségével.

Visszatérési típusa `NodeList`,
`Array` típusra az
`Array.from(allElems);`
segítségével konvertálható.



DOM - CSS szelektorok

- Elemi szelektorok:
  - Típus: `div`, `span`
  - Osztály: `.classname`
  - Azonosító: `#id`
  - Szűrt típus: `div.classname`, `span#id`
  - Attribútum: `div[attrib=value]`
  - Pszeudo-osztály: `div:hover`, `div:visited`, stb.
  - Pszeudo-elem: `div::before`, `div::after`
- Kombinált szelektorok
  - Leszármazott: `div span`
  - Gyerek: `div > span`
  - Későbbi testvér: `div ~ span`
  - Következő testvér: `div + span`

Forrás: *MDN Web Docs*

DOM - Attribútumok elérése

Legyen a HTML tag-ünk egy

```
<input value="42" data-id="1" readonly>
```

beviteli mező, amelyet le lehet kérdezni és tárolni a

```
const input = document.querySelector('input');
```

segítségével.

Az `input` változón keresztül lehetőség van a tag attribútumait olvasni

```
input.readOnly; // true
input.dataset.id; // "1"
```

és írni is

```
input.value = "1337";
```

DOM - CSS-attribútumok elérése

Legyen a HTML tag-ünk egy

```
<input class="bg-red td-bold" style="color: blue;">
```

beviteli mező az `input` változóban.

Az `input` változón keresztül lehetőség van a tag CSS-attribútumait olvasni és írni

```
input.style.color = "red";
```

illetve CSS-osztályait számos függvénnyel manipulálni

```
input.classList.add("bg-green");
input.classList.contains("bg-green"); // false
input.classList.toggle("bg-green"); // add or rem.
input.classList.remove("bg-red");
```

DOM - Tartalom elérése

Legyen a HTML tag-ünk egy

```
<span id="test">Szöveg</span>
```

szöveg, amelyet le lehet kérdezni és tárolni a

```
const span = document.querySelector('#test');
```

segítségével.

A `span` változón keresztül lehetőség van annak belső HTML-tartalmának olvasására

```
span.innerHTML; // "Szöveg"
```

és írására is

```
span.innerHTML = "Újabb szöveg";
```

DOM - Elemek hozzáadása és eltávolítása

Egy elem DOM reprezentációja létrehozható a

```
const elem = document.createElement("span");
elem.textContext = "Hello világ!";
```

segítségével.

A létrehozott elem hozzáadható

```
document.body.appendChild(elem);
```

majd el is eltávolítható az

```
elem.remove();
```

segítségével a DOM-ból.

Események

Az események egy, a felhasználók eset történetekből is ismert **WHEN-THEN** logikát követnek. Amikor (**WHEN**) egy bizonyos esemény bekövetkezik, akkor (**THEN**) egy bizonyos függvény lefut.

```
function handler() { /* THEN */ }
elem.addEventListener("type", handler); // WHEN
```

Események - Delegáció

Legyenek HTML tag-jeink a

```
<div> <input/><input/><input/> </div>
```

beviteli mezők. Tároljuk el azokat a szülőjükkel a

```
const div = document.querySelector("div");
```

segítségével.

Amikor egy eseménykezelőt egy bizonyos elem több, hasonló leszármazottjához kell hozzárendelni, olyankor delegálást használunk. Ekkor az eseménykezelő a szülő elemhez kerül hozzárendelésre.

```
delegate(div, "click", "input", handler);
    // szülő,    WHEN,  gyerek,    THEN
```

Események - Eseménytípus

```
elem.addEventListener("type", (event) => { /* THEN */ });
```

- `MouseEvent` - egéresemény
  - `"click"` - egérkattintás
  - `"mousemove"` - egérmozgatás
  - `"mousedown"`, `"mouseup"` - egér gombjának lenyomása, ill. felenegedése
  - `"scroll"` - görgetés az oldalon
- `KeyboardEvent` - billentyűzetesemény
  - `"keypress"` - billentyűzet gombjának megnyomása
  - `"keydown"`, `"keyup"` - billentyűzet gombjának lenyomása, ill. felengedése
- `InputEvent` - bevitelesemény
  - `"input"` - input mező értékének megváltozása
- `SubmitEvent` - beküldésesemény
  - `"submit"` - űrlap elküldése

Események - Eseményobjektum

```
elem.addEventListener("type", (event) => { /* THEN */ });
```

- `Event` - esemény
  - `type` - az esemény típusa (pl. `click`)
  - `target` - az esemény célpontja (lehet akár gyerek is)
  - `preventDefault()` - alapértelmezett viselkedés meggátlása
  - `stopPropagation()` - delegáláskori buborákoltatás megállítása
- `MouseEvent` - egéresemény
  - `clientX`, `clientY` - a kattintás X és Y koordinátája a lapon
  - `button` - a kattintás gombjai (0 = bal, 1 = görgő, 2 = jobb, stb.)
- `KeyboardEvent` - billentyűzetesemény
  - `key`, `code` - a lenyomott billentyű (pl. `w`, `KeyW` vagy `1`, `Digit1`)
- `InputEvent` - beviteleesemény
  - `data` - a beírt vagy beillesztett szöveg
  - `inputType` - a változtatás típusa (pl. `"insertText"`, `"insertFromPaste"`, `"deleteContentBackward"`)

Forrás: *MDN Web Docs*